FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Gavryl Garlitos
Section:	AN43
Professor:	Crisola G. Tan
Date:	October 29 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule
3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

Task 1:

```
facts = {
    "power_connected": True,
    "battery_charged": False,
    "power_light": False,
    "screen_visible": False,
```

```
    "fan_noise": False,
    "overheating": False,
    "wifi_connected": True,
    "temperature": 45
}

# Three categories of knowledge
categories = {
    "power_status": ["power_connected", "battery_charged", "power_light"],
    "display_status": ["screen_visible"],
    "hardware_status": ["fan_noise", "overheating", "temperature"],
    "network_status": ["wifi_connected"]
}

print("=" * 55)
print("PART 1: KNOWLEDGE REPRESENTATION")
print("=" * 55)

print("\nKnowledge Base Facts:")
for fact, value in facts.items():
    print(f"- {fact}: {value}")

print("\nKnowledge Categories:")
for category, items in categories.items():
    print(f"- {category}: {', '.join(items)}")
```

Task 2

```
def rule_power(facts):
    # IF the power cable is disconnected
    # THEN recommend connecting the power cable
    if not facts["power_connected"]:
        return "Connect the laptop to a power source."

    # IF the battery is not charged and the power light is off
    # THEN check the charger and power connection
    if not facts["battery_charged"] and not facts["power_light"]:
```

```
        return "Check the charger, power adapter, and battery."

    return None


def rule_display(facts):
    # IF the laptop has power but the screen is not visible
    # THEN troubleshoot the display
    if facts["power_light"] and not facts["screen_visible"]:
        return "Check the display brightness and external monitor."

    return None


def rule_overheating(facts):
    # IF the laptop is overheating
    # THEN improve cooling and check the ventilation
    if facts["overheating"] or facts["temperature"] >= 80:
        return "Clean the vents and allow the laptop to cool down."

    return None


def rule_wifi(facts):
    # IF Wi-Fi is disconnected
    # THEN check the wireless connection
    if not facts["wifi_connected"]:
        return "Check the Wi-Fi settings and reconnect to the network."

    return None


# Task 2B - Implement the Inference Engine
def inference_engine(facts):
    rules = [
        rule_power,
        rule_display,
```

```
        rule_overheating,
        rule_wifi
    ]

    actions = []

    # Check every rule against the knowledge base
    for rule in rules:
        result = rule(facts)
        if result is not None:
            actions.append(result)

    return actions


# Task 2C - Display RBR Output
print("\n" + "=" * 55)
print("PART 2: RULE-BASED REASONING (RBR)")
print("=" * 55)

rbr_actions = inference_engine(facts)

print("\nRule-Based Reasoning Actions:")

if rbr_actions:
    for action in rbr_actions:
        print(f"- {action}")
else:
    print("- No troubleshooting actions are needed.")
```

Task 3

```
case_base = [
    {
        "problem": {
            "power_connected": False,
            "battery_charged": False,
            "power_light": False,
            "screen_visible": False,
            "fan_noise": False,
```

```
            "overheating": False,
            "wifi_connected": True
        },
        "solution": "Connect the power cable and check the adapter."
    },
    {
        "problem": {
            "power_connected": True,
            "battery_charged": True,
            "power_light": True,
            "screen_visible": False,
            "fan_noise": True,
            "overheating": False,
            "wifi_connected": True
        },
            "solution": "Check the screen brightness, display cable, or
monitor."
    },
    {
        "problem": {
            "power_connected": True,
            "battery_charged": True,
            "power_light": True,
            "screen_visible": True,
            "fan_noise": True,
            "overheating": True,
            "wifi_connected": True
        },
        "solution": "Clean the cooling vents and check the laptop fan."
    },
    {
        "problem": {
            "power_connected": True,
            "battery_charged": True,
            "power_light": True,
            "screen_visible": True,
            "fan_noise": False,
```

```
            "overheating": False,
            "wifi_connected": False
        },
        "solution": "Enable Wi-Fi and reconnect to the network."
    }
]


# Task 3B - Retrieve the Most Similar Case
def calculate_similarity(new_problem, old_problem):
    # Count how many problem features match
    matching_features = 0

    for feature in new_problem:
        if new_problem[feature] == old_problem[feature]:
            matching_features += 1

    # Calculate similarity as a percentage
    total_features = len(new_problem)
    return matching_features / total_features


def retrieve_case(new_problem, case_base):
    best_case = None
    best_similarity = -1

    # Compare the new problem with every stored case
    for case in case_base:
        similarity = calculate_similarity(
            new_problem, case["problem"]
        )

        if similarity > best_similarity:
            best_similarity = similarity
            best_case = case

    return best_case, best_similarity
```

```
# Task 3C - Reuse the Retrieved Solution
# Task 3D - Revise the Solution
# Task 3E - Retain the New Case

print("\n" + "=" * 55)
print("PART 3: CASE-BASED REASONING (CBR)")
print("=" * 55)

print("\nStored Cases:", len(case_base))

print("\nEnter the current laptop symptoms.")
print("Answer using y for Yes or n for No.")

def ask_yes_no(question):
    while True:
        answer = input(question + " (y/n): ").strip().lower()

        if answer in ("y", "n"):
            return answer == "y"

        print("Invalid answer. Please enter y or n.")


# Gather a new problem from the user
new_problem = {
    "power_connected": ask_yes_no("Is the power cable connected?"),
    "battery_charged": ask_yes_no("Is the battery charged?"),
    "power_light": ask_yes_no("Is the power light on?"),
    "screen_visible": ask_yes_no("Is anything visible on the screen?"),
    "fan_noise": ask_yes_no("Can you hear the fan?"),
    "overheating": ask_yes_no("Is the laptop overheating?"),
    "wifi_connected": ask_yes_no("Is Wi-Fi connected?")
}

# RETRIEVE: Find the most similar past case
```

```
matched_case, similarity = retrieve_case(
    new_problem, case_base
)

print("\n--- 4R CYCLE ---")
print("\n1. RETRIEVE")
print("Most similar past case found.")
print(f"Similarity score: {similarity:.2%}")

# REUSE: Apply the past solution to the new problem
suggested_solution = matched_case["solution"]

print("\n2. REUSE")
print("Suggested solution:", suggested_solution)

# REVISE: Let the user accept or change the solution
print("\n3. REVISE")
print("You may accept the suggested solution or revise it.")

while True:
    choice = input(
        "Keep this solution? (y/n): "
    ).strip().lower()

    if choice in ("y", "n"):
        break

    print("Invalid answer. Please enter y or n.")

if choice == "y":
    final_solution = suggested_solution
    print("Solution accepted.")
else:
    final_solution = input(
        "Enter your revised troubleshooting solution: "
    ).strip()
```

```
    if not final_solution:
        final_solution = suggested_solution
        print("Empty solution. Original solution retained.")

# RETAIN: Save the new problem and its final solution
new_case = {
    "problem": new_problem.copy(),
    "solution": final_solution
}

case_base.append(new_case)

print("\n4. RETAIN")
print("The new problem and its solution were saved.")
print("Updated number of stored cases:", len(case_base))

print("\nFinal Recommended Solution:")
print(final_solution)

print("\n" + "=" * 55)
print("PROGRAM COMPLETED")
print("=" * 55)
```

Sample Output

```
Stored Cases: 4

Enter the current laptop symptoms.
Answer using y for Yes or n for No.
Is the power cable connected? (y/n): y
Is the battery charged? (y/n): y
Is the power light on? (y/n): y
Is anything visible on the screen? (y/n): n
Can you hear the fan? (y/n): n
Is the laptop overheating? (y/n): y
Is Wi-Fi connected? (y/n): y

--- 4R CYCLE ---

1. RETRIEVE
Most similar past case found.
Similarity score: 71.43%

2. REUSE
Suggested solution: Check the screen brightness, display cable, or monitor.

3. REVISE
You may accept the suggested solution or revise it.
Keep this solution? (y/n): y
Solution accepted.

4. RETAIN
The new problem and its solution were saved.
Updated number of stored cases: 5

Final Recommended Solution:
Check the screen brightness, display cable, or monitor.

==========================================================
PROGRAM COMPLETED
==========================================================
```